

Rapport TP Modele discret

DOUIH Zakaria
NAITELHAJ Abderrahmane
RAJI Ahmed

December 15, 2025

Problème de localisation et d'allocation

Exercice 1

Enoncé

Une entreprise marocaine de distribution de produits cosmétiques cherche à s'établir pour la première fois dans une ville subsaharienne. La ville compte un ensemble I de détaillants de produits cosmétiques, dispersés sur le territoire de la ville. Chaque détaillant $i \in I$ dessert une population de taille p_i . Après plusieurs investigations, l'entreprise a pu repérer un ensemble S d'entrepôts disponibles pour location. Le temps moyen de parcours entre un détaillant $i \in I$ et un entrepôt $s \in S$ est dénoté t_{si} . On suppose que la clientèle d'un détaillant est desservi par un entrepôt si le temps de parcours entre l'entrepôt et le détaillant est au plus $T = 45$ minutes.

En se basant sur une étude du marché, un bureau d'études a recommandé à l'entreprise de ne louer que 2 entrepôts dans la ville en question dans une première étape.

Question 1 : Formuler le problème comme un PNE de telle sorte à maximiser le nombre total de clients desservis (Modèle M1)

Définissons les variables de décision :

- $x_s = 1$ si l'entrepôt s est loué, 0 sinon
- $y_{si} = 1$ si la clientèle i est asservi par l'entrepôt s , 0 sinon

Listons les données :

- t_{si} la durée moyenne entre l'entrepôt s et le détaillant i
- $g_{si} = 1$ si la durée $t_{si} \leq 45$, 0 sinon
- p_i la taille de la clientèle i

Voici le programme linéaire à résoudre :

$$\text{Max} \sum_{i \in I, s \in S} y_{si} p_i \quad (1)$$

$$\text{s.c.} \sum_{s \in S} y_{si} \leq 1, \forall i \in I \quad (2)$$

$$\sum_{i \in I} y_{si} \leq n x_s, \forall s \in S \quad (3)$$

$$\sum_{s \in S} x_s \leq 2 \quad (4)$$

$$y_{si} \leq g_{si}, \forall s \in S, \forall i \in I \quad (5)$$

$$x_s, y_{si}, g_{si} \in \{0, 1\}, \forall s \in S, \forall i \in I \quad (6)$$

$$p_i \in N \quad (7)$$

- (1) : On veut maximiser la population asservi par l'entreprise
- (2) : Chaque population est asservi par un entrepôt **au plus**
- (3) : Si un entrepôt n'est pas alloué, alors il ne serve aucune clientèle
- (4) : On ne peut louer que 2 entrepôts au plus
- (5) : Un entrepôt dessert un détaillant seulement si la durée de parcours est inférieure à 45min

Question 2 : Implémenter et résoudre le problème avec CPLEX
 Commençant avec le code CPLEX :

```

1 //ensembles
2 int tailleI = ...;
3 range I = 1..tailleI;
4 int tailleS = ...;
5 range S = 1..tailleS;
6
7 //donnees
8 int t[I][S] = ...;
9 int p[I] = ...;
10 int g[I][S];
11 int K = ...;
12 float T = ...;
13 //variables de decision
14 dvar boolean y[I][S];
15 dvar boolean x[S];
16 //definition de g
17 execute {
18     for(var entrepot in S){
19         for(var detaillnant in I){
20             if(t[detaillnant][entrepot] <= T)
21                 {

```

```

22                                     g[detaillant][entrepot] =
23                                     1;
24                                     }
25                                     else{
26                                     g[detaillant][entrepot] = 0;
27                                     }
28                                     }
29                                     }
30 }
31 //probleme math.
32 maximize sum ( i in I , s in S ) p[i]* y[i,s];//1
33 subject to{
34     forall ( i in I )
35         sum ( s in S ) y [i][s] <= 1;//2
36     forall ( s in S , i in I )
37         y [i][s] <= x [s];//3
38     sum ( s in S ) x [ s ] <= K ;//4
39     forall(i in I, s in S)
40         y[i][s] <= g[i][s];//5
41 }

```

- T dans set exemple de l'exercice est égale à 45
- K dans cet exemple de l'exercice est égale à 2

N.B. Le fichier fourni, i.e. TP1.Ex1.txt n'est pas adéquat avec le format de Cplex. Pour résoudre le problème, nous proposons un code C qui permet d'extraire les matrices que l'on a besoin en le bon format

```

1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <string.h>
4
5      #define MAX_LINE_LENGTH 1024
6      #define ROWS 100
7      #define COLUMNS 10
8
9      void extract_column_and_matrix(const char *file_path) {
10         FILE *file = fopen(file_path, "r");
11         if (file == NULL) {
12             perror("Erreur lors de l'ouverture du fichier");
13             return;
14         }
15
16         char line[MAX_LINE_LENGTH];
17         float column_data[ROWS];
18         float matrix[ROWS][COLUMNS];
19

```

```

20     int row = 0;
21
22     while (fgets(line, sizeof(line), file) && row < ROWS) {
23         // Supprimer le saut de ligne
24         line[strcspn(line, "\n")] = 0;
25
26         // Diviser la ligne en colonnes avec \t comme
27         // delimitateur
28         char *token = strtok(line, "\t");
29         int col = 0;
30
31         // Lire la deuxieme colonne (index 1)
32         for (int i = 0; token != NULL; i++) {
33             if (i == 1) {
34                 column_data[row] = atof(token);
35             } else if (i > 1 && i <= 11) { // Remplir la
36                 // matrice a partir de la troisieme colonne
37                 matrix[row][col++] = atof(token);
38             }
39             token = strtok(NULL, "\t");
40         }
41         row++;
42     }
43
44     fclose(file);
45
46     // Afficher les resultats
47     printf("float column_data[%d] = [", row);
48     for (int i = 0; i < row; i++) {
49         printf("%d%s", (int)column_data[i], i < row - 1 ? ", " : "");
50     }
51     printf("]\n\n");
52
53     printf("float matrix[%d][%d] = [\n", row, COLUMNS);
54     for (int i = 0; i < row; i++) {
55         printf("    [ ");
56         for (int j = 0; j < COLUMNS; j++) {
57             printf("%d%s", (int)matrix[i][j], j < COLUMNS -
58                 1 ? ", " : "");
59         }
60         printf(" ]%s\n", i < row - 1 ? ", " : "");
61     }
62     printf("]\n");
63 }
64
65 int main() {
66     const char *file_path = "C:\\Users\\douih\\OneDrive\\
67         Bureau\\insea\\md\\TP1_Ex1.txt"; // Remplacez par le
68         chemin de votre fichier

```

```

64     extract_column_and_matrix(file_path);
65     return 0;
66 }

```

Le fichier données contient les éléments suivants :

```

1     tailleI = 100;
2     tailleS = 10;
3     K = 2;
4     T = 45;
5     p = [167, 438, 333, 124, 122, 96, 269, 134, 414, 168,
6          275, 271, 265, 153, 248, 250, 159, 472, 179, 144,
7          108, 319, 299, 206, 253, 459, 159, 90, 496, 454,
8          173, 345, 325, 217, 214, 485, 263, 421, 88, 225,
9          167, 470, 485, 497, 237, 316, 293, 133, 116, 468,
10         436, 320, 438, 192, 260, 209, 68, 490, 467, 475,
11         54, 173, 365, 170, 338, 399, 111, 188, 280, 484,
12         187, 335, 490, 309, 468, 395, 76, 161, 423, 83,
13         210, 248, 232, 471, 83, 165, 106, 457, 441, 462,
14         336, 138, 92, 180, 95, 499, 272, 211, 235, 312];
15
16     t = [
17         [ 7, 54, 59, 48, 32, 31, 9, 59, 27, 51 ],
18         [ 19, 23, 22, 46, 47, 24, 16, 23, 8, 37 ],
19         [ 56, 44, 43, 44, 52, 5, 20, 58, 26, 18 ],
20         [ 34, 32, 55, 33, 54, 55, 59, 43, 29, 55 ],
21         [ 58, 10, 28, 33, 8, 5, 19, 18, 45, 30 ],
22         [ 13, 27, 53, 22, 59, 10, 7, 32, 6, 51 ],
23         [ 8, 53, 8, 33, 36, 45, 33, 45, 36, 37 ],
24         [ 49, 45, 11, 22, 43, 38, 60, 26, 41, 37 ] ... etc

```

Les résultats sous CPLEX sont les suivants :

```

// solution (optimal) with objective 27242
// Quality Incumbent solution:
// MILP objective                2.72420000e
// MILP solution norm |x| (Total, Max) 9.900000e+01
// MILP solution error (Ax=b) (Total, Max) 0.000000e+00
x = [0 0 1 0 0 0 0 1 0 0];

```

Interprétation :

Donc l'entreprise alloue les entrepôts 3 et 8. La fonction objective a comme valeur 27242.

Une nouvelle étude du marché a montré que chaque client desservi rapporterait un profit mensuel moyen de 6 dollars à l'entreprise, et que les loyers mensuels des 10 entrepôts sont connus.

Question 3 : Ajuster le modèle pour le rendre capable de définir le nombre et les localisations optimales d'entrepôts à louer afin de maximiser le profit de l'entreprise (Modèle M2).

Dans cette partie, il suffit de modifier dans la fonction objective. On veut atteindre deux objectifs :

- Maximiser le profit (simplement multiplier par 6)
- Minimiser les frais de location (soustraire les coûts) notés $f_s, \forall s \in S$
- On omet la contrainte (4)

La nouvelle fonction objectif s'écrit comme suit :

$$\text{Max } 6 \sum_{i \in I, s \in S} y_{si} p_i - \sum_{s \in S} x_s f_s \quad (8)$$

Question 4 : Implémenter et résoudre le modèle avec CPLEX

On doit ajouter la matrice des prix de locations au code source de CPLEX. On ajoute dans la partie déclaration :

```
1 float f[S] = ...;
```

et dans la fonction objective :

```
1 maximize 6*sum ( i in I , s in S ) p[i]* y[i,s] - sum(s
    in S) x[s]*f[s];
```

Dans le fichier de données :

```
1 f = [4596, 2570, 2989, 3799, 3147, 2490, 3596, 1158,
      2356, 2238];
```

Les résultats sous CPLEX sont les suivants :

```
// solution (optimal) with objective 159554
// Quality Incumbent solution:
// MILP objective 1.59554
// MILP solution norm |x| (Total, Max) 1.03000
x = [0 0 0 0 0 0 0 0 1 1];
```

Interprétation :

Donc l'entreprise alloue les entrepôts 8, 9 et 10. La fonction objective a comme valeur 159554 DH.

Question 5 : Trouver un moyen pour comparer les résultats des deux modèles M1 et M2, en tirant profit des données de la nouvelle étude. Conclure.

Pour comparer les deux résultats on peut raisonner sur le nombre des clients asservis. Pour ce faire on doit chercher dans la matrice y et sommer les tailles de la clientèle.

Il y existe beaucoup de façons à le faire, mais la méthode la plus simple est de travailler avec le bloc d'exécution "execute" après la définition du problème :

```

1 execute{
2   var c=0;
3   for(var i in I){
4     for(var s in S){
5       c += y[i][s]*p[i];
6     }
7   }
8   writeln("La somme de population aservie est : "+c);
9 }

```

On compare maintenant les résultats.

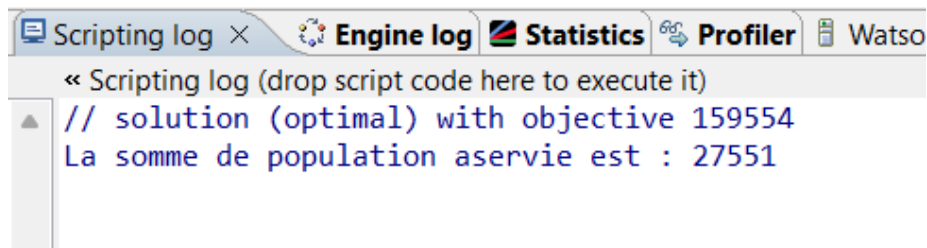


Figure 3: Population en M2

La fonction objective du M1 donne déjà le nombre de population : $27242 \leq 27551$. **Conclusion :** M2 serve plus de clients que M1. Alors

Exercice 2

Enoncé

Sur la base des résultats du modèle M2, l'entreprise a loué un ensemble d'entrepôts $E \subseteq S$ et a décidé de desservir chaque détaillant depuis l'entrepôt le plus proche. les gestionnaires se sont heurtés à des problèmes de gestion en raison de la croissance de la demande et de l'élargissement de la gamme de produits proposés. Un expert en optimisation a recommandé de revoir ce plan d'approvisionnement des détaillants en promettant des gains. L'entreprise a ainsi collecté les nouvelles données suivantes :

- P : Ensemble de produits vendus dans la ville cible.
- d_i^p : Demande de chaque détaillant $i \in I$ du produit $p \in P$
- q_e Quantité de produits que peut fournir un entrepôt $e \in E$
- c_{ei} Coût de transporter une unité d'un produit de l'entrepôt $e \in E$ au détaillant $i \in I$

- f_{ei} : Coût fixe à payer si un ou plusieurs produits sont acheminés de l'entrepôt $e \in E$ au détaillant $i \in I$

Variables de décision

- x_{ei} : si l'entrepôt e dessert le détaillant i
- y_{ei}^p : la quantité transporté de l'entrepôt e au client i du produit p

Problème mathématique

$$\text{Min} \sum_{e \in P} \sum_{e \in E} \sum_{i \in I} c_{ei} y_{ei}^p + \sum_{e \in E} \sum_{i \in I} f_{ei} x_{ei} \quad (9)$$

$$\text{s.c. } y_{ei}^p \leq M \cdot x_{ei}, \forall e \in E, \forall i \in I, \forall p \in P \quad (10)$$

$$\sum_{e \in E} y_{ei}^p = d_i^p, \forall p \in P, \forall i \in I \quad (11)$$

$$\sum_{i \in I} \sum_{p \in P} y_{ei}^p \leq q_e, \forall e \in E \quad (12)$$

(10) On minimise les coûts fixe et les coûts de transport

(11) si un produit est transporté de l'entrepôt e au détaillant i , l'entrepôt e doit desservir le détaillant i

(12) La demande des détaillants doit être satisfaite

(13) Un entrepôt une capacité de production à respecter

(14) Un détaillant travaille avec les entrepôts proches

Question 2 : Le modèle M3 comporte-il un grand-M ? Si oui, donnez-lui la valeur la plus faible valeur admissible.

Le modèle M3 comporte d'un grand-M (la contrainte 11). La valeur la plus faible est la demande maximale dans la matrice d_i^p : $\max_{i \in I, p \in P} d_i^p$.

Question 3 : Implémenter le modèle M3 et le résoudre

N.B. Le fichier TP1_Ex2_corrected.txt contient des données non conformes au syntaxe OPL. Pour ce faire, on peut travailler avec ce code python et modifier un peu sur le fichier (le fichier final est inclus dans les fichiers fournies avec ce rapport

```

1 data = """la data"""
2 # Split the data into lines
3 lines = data.splitlines()
4
5 # Initialize the result list
6 result = []

```

```

7
8 # Process each line
9 for line in lines:
10     # Split the line by spaces and remove empty strings
11     values = [value for value in line.split(' ') if value]
12     # Extract the numerical values and convert them to floats
13     result.append([float(value) for value in values[1:]])
14
15 # Print the result
16 print(result)

```

Code CPLEX :

```

1  int tailleI = ...;
2  range I = 1..tailleI;
3  int tailleE = ...;
4  range E = 1..tailleE;
5  int tailleP = ...;
6  range P = 1..tailleP;
7  //donnees
8  float c[E][I] = ...;
9  float f[E][I] = ...;
10 int d[I][P] = ...;
11 int M = max(i in I, p in P) d[i][p];
12 int q[E] = ...;
13 //variables de decision
14 dvar int+ y[I][E][P];
15 dvar boolean x[I][E];
16 execute{
17     writeln(M);
18 }
19 //probleme math.
20 minimize sum(i in I, e in E, p in P) y[i][e][p]*c[e][i] +
21     sum(i in I, e in E) x[i][e]*f[e][i];//9
22 subject to{
23     forall(e in E, i in I, p in P)
24         y[i][e][p] <= M*x[i][e];//10
25     forall(p in P, i in I)
26         sum(e in E) y[i][e][p] == d[i][p];//11
27     forall(e in E)
28         sum(i in I, p in P) y[i][e][p] <= q[e];//12
29 }

```

Question 4 : En supposant que chaque détaillant doit être approvisionné à partir d'un seul entrepôt pour chaque produit $p \in P$, reformuler le problème (Modèle M4).

On ajoute une variable de décision z_{ei}^p qui est égale à 1 si l'entrepôt e dessert le détaillant i par le produit p , 0 sinon.

```

// solution (optimal) with objective 72926.19
// Quality Incumbent solution:
// MILP objective 7.2926190000e+04
// MILP solution norm ||x|| (Total, Max) 2.51580e+04 1.00000e+02
// MILP solution error (feas) (Total, Max) 0.00000e+00 0.00000e+00
// MILP x bound error (Total, Max) 7.46679e-13 6.40899e-13
// MILP x integrality error (Total, Max) 1.38764e-12 6.40932e-13
// MILP slack bound error (Total, Max) 3.73339e-10 6.40899e-11
//
y = [[[0
      0 0 0 0]
      [0 0 0 0]
      [22 9 75 36 31]]
      [[72 61 27 44 2]
      [0 0 0 0]
      [0 0 0 0 0]]
      [[0 0 0 0]
      [33 100 73 22 46]
      [0 0 0 0 0]]
      [[0 0 0 0]
      [60 31 5 72 28]
      [0 0 0 0 0]]
      [[0 0 0 0]
      [59 61 10 34 75]
      [0 0 0 0 0]]]]

```

On ajoute la contrainte :

$$\sum_{e \in E} z_{ei}^p = 1, \forall p \in P, \forall i \in I \quad (13)$$

et on modifie dans la contrainte 10 :

$$y_{ei}^p \leq M \cdot z_{ei}^p, \forall e \in E, \forall i \in I, \forall p \in P \quad (14)$$

On ajoute la contrainte :

$$\sum_{p \in P} z_{ei}^p \leq |P| x_{ei}, \forall i \in I, \forall e \in E \quad (15)$$

Question 5 : Le modèle M4 comporte-il un grand-M ? Si oui, donnez-lui la valeur la plus faible valeur admissible.

Le modèle contient le grand-M dans la contrainte 16. La valeur de M est la même : $\max_{i \in I, p \in P} d_i^p$.

Question 6 : Implémenter le modèle M4 et le résoudre.

Code CPLEX :

```

1 int tailleI = ...;
2 range I = 1..tailleI;
3 int tailleE = ...;
4 range E = 1..tailleE;
5 int tailleP = ...;
6 range P = 1..tailleP;
7 //donnees
8 float c[E][I] = ...;
9 float f[E][I] = ...;
10 int d[I][P] = ...;
11 int M = max(i in I, p in P) d[i][p];
12 int q[E] = ...;
13 //variables de decision
14 dvar int+ y[I][E][P];
15 dvar boolean x[I][E];
16 dvar boolean z[I][E][P];
17 execute{

```

```

18     writeln(M);
19 }
20 //probleme math.
21 minimize sum(i in I, e in E, p in P) y[i][e][p]*c[e][i] +
22     sum(i in I, e in E) x[i][e]*f[e][i]; //9
23 subject to{
24     forall(e in E, i in I, p in P) y[i][e][p] <= M*x[i][
25         e]; //14
26     forall(p in P, i in I) sum(e in E) y[i][e][p] == d[i
27         ][p]; //11
28     forall(e in E) sum(i in I, p in P) y[i][e][p] <= q[e
29         ]; //12
30     forall(i in I, p in P) sum(e in E) z[i][e][p] == 1;
31         //13
32     forall(i in I, e in E) sum(p in P) z[i][e][p] <=
33         tailleP*x[i][e]; //15
34 }

```

```

// solution (optimal) with objective 72926.19
// Quality Incumbent solution:
// MILP objective                7.2926190000e+04
// MILP solution norm |x| (Total, Max)  2.56580e+04  1.00000e+02
// MILP solution error (Ax=b) (Total, Max)  0.00000e+00  0.00000e+00
// MILP x bound error (Total, Max)  7.04994e-12  3.20455e-12
// MILP x integrality error (Total, Max)  1.41000e-11  3.20455e-12
// MILP slack bound error (Total, Max)  3.23654e-10  6.40899e-11
,,

```

Question 7 : En supposant que pour des raisons de gestion d'inventaire, un entrepôt ne peut pas desservir plus que 3 produits, Modifier le modèle 4 pour intégrer cette nouvelle restriction (Modèle M5).

Pour ce faire, on va ajouter une variable de décision r_e^p binaire qui détermine si le produit est desservi par l'entrepôt ou non.

Il en résulte une relation entre cette variable et les autres variables, ce qui est décrit dans les contraintes suivantes à ajouter :

$$\sum_{e \in E} r_e^p \leq 3, \forall p \in P \quad (16)$$

$$z_{ei}^p \leq r_e^p, \forall p \in P, \forall i \in I, \forall e \in E \quad (17)$$

$$r_e^p \leq \sum_{i \in I} z_{ei}^p, \forall p \in P, \forall e \in E \quad (18)$$

Question 8 : Implémenter M5 et résoudre

Code CPLEX :

```

1  int tailleI = ...;
2  range I = 1..tailleI;
3  int tailleE = ...;

```

```

4 range E = 1..tailleE;
5 int tailleP = ...;
6 range P = 1..tailleP;
7 //donnees
8 float c[E][I] = ...;
9 float f[E][I] = ...;
10 int d[I][P] = ...;
11 int M = max(i in I, p in P) d[i][p];
12 int q[E] = ...;
13 //variables de decision
14 dvar int+ y[I][E][P];
15 dvar boolean x[I][E];
16 dvar boolean z[I][E][P];
17 dvar boolean r[E][P];
18 execute{
19     writeln(M);
20 }
21 //probleme math.
22 minimize sum(i in I, e in E, p in P) y[i][e][p]*c[e][i] +
23     sum(i in I, e in E) x[i][e]*f[e][i];
24 subject to{
25     forall(e in E, i in I, p in P) y[i][e][p] <= M*x[i][e]; //14
26     forall(p in P, i in I) sum(e in E) y[i][e][p] == d[i][p]; //11
27     forall(e in E) sum(i in I, p in P) y[i][e][p] <= q[e]; //12
28     forall(i in I, p in P) sum(e in E) z[i][e][p] == 1; //13
29     forall(i in I, e in E) sum(p in P) z[i][e][p] <=
30         tailleP*x[i][e]; //15
31     forall(e in E, i in I, p in P) z[i][e][p] <= r[e][p]; //17
32     forall(e in E, p in P) r[e][p] <= sum(i in I) z[i][e][p]; //18
33     forall(e in E) sum(p in P) r[e][p] <= 3; //16
34 }

```

N.B. chaque entrepôt ne peut desservir que 3 produits (3 types de produits).

Question 9 : Compte tenu des données disponibles, y a-t-il une autre opportunité d'optimisation manquée pour ce problème ? expliquer.

0.1 Exercice 3

Une boulangerie s'est engagé à assurer la livraison du pain à ses 5 clients potentiels situés dans son voisinage. Elle dispose de 3 triporteurs électriques, et souhaite optimiser ses tournées de livraison de pain. L'objectif est de minimiser

```

// solution (integer optimal, tolerance) with objective 78535.2
// Quality Incumbent solution:
// MILP objective 7.8535200000e+04
// MILP solution norm |x| (Total, Max) 2.57660e+04 1.00000e+02
// MILP solution error (Ax=b) (Total, Max) 1.33227e-15 3.33067e-16
// MILP x bound error (Total, Max) 6.66134e-16 2.22045e-16
// *****

[[0 0 0 0]
 [0 1 1 0 0]
 [1 0 0 1 1]]
[[0 1 1 1 0]
 [0 0 0 0 0]
 [1 0 0 0 1]]
[[0 1 1 0 0]
 [0 0 0 0 0]
 [1 0 0 1 1]]];
r = [[0 1 1 1 0]
      [0 1 1 0 0]
      [1 0 0 1 1]];

```

le coût total des circuits tout en respectant les capacités des triporteurs et les demandes des clients.

Question 1 : Modéliser le problème en nombres entiers.

Données :

- n : clients à visiter avec demande d_i - $N = \{1, \dots, n\}$
- K : ensemble de triporteurs disponibles
- Q : capacité d'un triporteur $N^o = N \cup \{o\}$
- c_{ij} : distance du lieu i au lieu j (lieu = client ou dépôt)

Variables de décision :

- x_{ij}^k 1 si le triporteur k visite le lieu j immédiatement après le lieu i , 0 sinon

Problème mathématique :

$$\min \sum_{k \in K} \sum_{i \in N^o} \sum_{j \in N^o, j \neq i} c_{ij} x_{ij}^k \quad (19)$$

$$s.c. \sum_{k \in K} \sum_{j \in N^o, j \neq i} x_{ij}^k = 1, \forall i \in N \quad (20)$$

$$\sum_{j \in N} x_{oj}^k \leq 1, \forall k \in K \quad (21)$$

$$\sum_{k \in K} \sum_{j \in N^o, j \neq i} x_{ij}^k - \sum_{k \in K} \sum_{j \in N^o, j \neq i} x_{ji}^k = 0, \forall k \in K, i \in K \quad (22)$$

$$\sum_{i \in N} \sum_{k \in K} \sum_{j \in N^o, j \neq i} d_i x_{ij}^k \leq Q, \forall k \in K \quad (23)$$

$$x_{ij}^k \in \{0, 1\}, \forall k \in K, i, j \in N^o, i \neq j \quad (24)$$

Question 2 : Implémenter et résoudre avec CPLEX.

```

1 // Declarations
2 setof(int) Clients = {1, 2, 3, 4, 5}; // Clients
3 setof(int) positions = {0, 1, 2, 3, 4, 5}; // positions
4 setof(int) Vehicles = {1, 2, 3}; // Vehicules
5 float Capacity[Vehicles] = [250, 300, 200]; // Capacites des
   vehicules
6 float Demand[Clients] = [100, 150, 200, 100, 50]; //
   Demandes des clients
7 float Cost[0..5][0..5] = [
8   [0, 12, 15, 20, 10, 18], // Matrice des couts
9   [12, 0, 10, 15, 5, 8],
10  [15, 10, 0, 8, 7, 12],
11  [20, 15, 8, 0, 12, 15],
12  [10, 5, 7, 12, 0, 6],
13  [18, 8, 12, 15, 6, 0]
14 ];
15
16 // Variables de decision
17 dvar boolean x[0..5][0..5][Vehicles]; // 1 si le vehicule
   parcourt (i, j)
18
19 // Fonction objectif
20 minimize sum(k in Vehicles, i in 0..5, j in 0..5 : i != j)
   Cost[i][j] * x[i][j][k];
21
22 // Contraintes
23 subject to {
24   // Chaque client est visite une fois
25   forall(i in Clients)
26     sum(k in Vehicles, j in 0..5 : j != i) x[i][j][k] == 1;

```

```

27
28 // Flux entrant et sortant equilibre
29 forall(k in Vehicles, i in 0..5)
30     sum(j in 0..5 : j != i) x[i][j][k] == sum(j in 0..5 : j
31         != i) x[j][i][k];
32
33 // Capacite des vehicules
34 forall(k in Vehicles)
35     sum(i in Clients) Demand[i] * sum(j in 0..5 : j != i) x[
36         i][j][k] <= Capacity[k];
37 }

```

```

// solution (optimal) with objective 68
Coût total : 68
Vehicule 1: 2 -> 4
Vehicule 1: 4 -> 2
Vehicule 2: 3 -> 5
Vehicule 2: 5 -> 3
Vehicule 3: 0 -> 1
Vehicule 3: 1 -> 0

```

Optimisation des tournées de livraison de pain

Modélisation du problème en nombres entiers

Nous modélisons le problème de tournées de véhicules comme un problème d'optimisation en nombres entiers. Ce modèle inclut les variables de décision, la fonction objectif et les contraintes nécessaires pour garantir une solution réalisable.

Définition des variables

- x_{ijk} : variable binaire qui prend la valeur 1 si le véhicule k effectue le trajet entre le client i et le client j , et 0 sinon, où $i, j \in \{0, 1, \dots, 5\}$ et $k \in \{1, 2, 3\}$.
- y_{ik} : variable binaire qui prend la valeur 1 si le client i est desservi par le véhicule k , et 0 sinon, où $i \in \{1, 2, \dots, 5\}$ et $k \in \{1, 2, 3\}$.

Paramètres

- c_{ij} : coût de parcours entre les clients i et j , incluant la boulangerie (B notée comme $i = 0$).
- d_i : demande du client i , en nombre de pains.
- Q_k : capacité maximale du triporteur k .

Fonction objectif

L'objectif est de minimiser le coût total des trajets effectués par les véhicules :

$$\text{Minimiser } \sum_{k=1}^3 \sum_{i=0}^5 \sum_{j=0}^5 c_{ij} \cdot x_{ijk}.$$

Contraintes

1. Chaque client doit être desservi par exactement un véhicule :

$$\sum_{k=1}^3 y_{ik} = 1, \quad \forall i \in \{1, \dots, 5\}.$$

2. Capacité des véhicules : la somme des demandes des clients desservis par un véhicule ne doit pas dépasser sa capacité :

$$\sum_{i=1}^5 d_i \cdot y_{ik} \leq Q_k, \quad \forall k \in \{1, 2, 3\}.$$

3. Flux équilibré pour chaque véhicule : si un véhicule entre dans un client i , il doit également en sortir :

$$\sum_{j=0}^5 x_{ijk} = \sum_{j=0}^5 x_{jik}, \quad \forall i \in \{1, \dots, 5\}, \forall k \in \{1, 2, 3\}.$$

4. Relation entre x_{ijk} et y_{ik} : un véhicule dessert un client uniquement si une tournée est assignée :

$$\sum_{j=0}^5 x_{ijk} = y_{ik}, \quad \forall i \in \{1, \dots, 5\}, \forall k \in \{1, 2, 3\}.$$

5. Départ et retour à la boulangerie pour chaque véhicule :

$$\sum_{j=1}^5 x_{0jk} = 1 \quad \text{et} \quad \sum_{i=1}^5 x_{i0k} = 1, \quad \forall k \in \{1, 2, 3\}.$$

6. Pas de boucle interne : un véhicule ne peut pas visiter un client et revenir immédiatement au même endroit :

$$x_{iik} = 0, \quad \forall i \in \{0, \dots, 5\}, \forall k \in \{1, 2, 3\}.$$

Résumé du modèle

$$\begin{aligned}
& \text{Minimiser} && \sum_{k=1}^3 \sum_{i=0}^5 \sum_{j=0}^5 c_{ij} \cdot x_{ijk} \\
& \text{sous contraintes} && \sum_{k=1}^3 y_{ik} = 1, \quad \forall i \in \{1, \dots, 5\}, \\
& && \sum_{i=1}^5 d_i \cdot y_{ik} \leq Q_k, \quad \forall k \in \{1, 2, 3\}, \\
& && \sum_{j=0}^5 x_{ijk} = \sum_{j=0}^5 x_{jik}, \quad \forall i \in \{1, \dots, 5\}, \forall k \in \{1, 2, 3\}, \\
& && \sum_{j=0}^5 x_{ijk} = y_{ik}, \quad \forall i \in \{1, \dots, 5\}, \forall k \in \{1, 2, 3\}, \\
& && \sum_{j=1}^5 x_{0jk} = 1 \quad \text{et} \quad \sum_{i=1}^5 x_{i0k} = 1, \quad \forall k \in \{1, 2, 3\}, \\
& && x_{iik} = 0, \quad \forall i \in \{0, \dots, 5\}, \forall k \in \{1, 2, 3\}.
\end{aligned}$$

Énumération des combinaisons de clients

Voici la liste des 31 combinaisons possibles de clients, sans dépasser les capacités des triporteurs. Chaque combinaison représente un groupe de clients qui pourrait être assigné à une même route, selon la capacité d'un triporteur.

- Combinaison 1 : ('C1',)
- Combinaison 2 : ('C2',)
- Combinaison 3 : ('C3',)
- Combinaison 4 : ('C4',)
- Combinaison 5 : ('C5',)
- Combinaison 6 : ('C1', 'C2')
- Combinaison 7 : ('C1', 'C3')
- Combinaison 8 : ('C1', 'C4')
- Combinaison 9 : ('C1', 'C5')
- Combinaison 10 : ('C2', 'C3')
- Combinaison 11 : ('C2', 'C4')
- Combinaison 12 : ('C2', 'C5')

- Combinaison 13 : ('C3', 'C4')
- Combinaison 14 : ('C3', 'C5')
- Combinaison 15 : ('C4', 'C5')
- Combinaison 16 : ('C1', 'C2', 'C3')
- Combinaison 17 : ('C1', 'C2', 'C4')
- Combinaison 18 : ('C1', 'C2', 'C5')
- Combinaison 19 : ('C1', 'C3', 'C4')
- Combinaison 20 : ('C1', 'C3', 'C5')
- Combinaison 21 : ('C1', 'C4', 'C5')
- Combinaison 22 : ('C2', 'C3', 'C4')
- Combinaison 23 : ('C2', 'C3', 'C5')
- Combinaison 24 : ('C2', 'C4', 'C5')
- Combinaison 25 : ('C3', 'C4', 'C5')
- Combinaison 26 : ('C1', 'C2', 'C3', 'C4')
- Combinaison 27 : ('C1', 'C2', 'C3', 'C5')
- Combinaison 28 : ('C1', 'C2', 'C4', 'C5')
- Combinaison 29 : ('C1', 'C3', 'C4', 'C5')
- Combinaison 30 : ('C2', 'C3', 'C4', 'C5')
- Combinaison 31 : ('C1', 'C2', 'C3', 'C4', 'C5')

Vérification des combinaisons réalisables

Nous avons évalué la faisabilité de chaque combinaison de clients en fonction des capacités des triporteurs et des demandes des clients. Voici le résultat de cette vérification :

Combinaisons réalisables

[label=●]('C1',) ('C2',) ('C3',) ('C4',) ('C5',) ('C1', 'C2') ('C1', 'C3') ('C1', 'C4') ('C1', 'C5') ('C2', 'C4') ('C2', 'C5') ('C3', 'C4') ('C3', 'C5') ('C4', 'C5') ('C1', 'C2', 'C5') ('C1', 'C4', 'C5') ('C2', 'C4', 'C5')

Combinaisons irréalisables

[label=●]('C2', 'C3') ('C1', 'C2', 'C3') ('C1', 'C2', 'C4') ('C1', 'C2', 'C5')
('C1', 'C3', 'C4') ('C1', 'C3', 'C5') ('C2', 'C3', 'C4') ('C2', 'C3', 'C5')
('C3', 'C4', 'C5') ('C1', 'C2', 'C3', 'C4') ('C1', 'C2', 'C3', 'C5') ('C1',
'C2', 'C4', 'C5') ('C1', 'C3', 'C4', 'C5') ('C2', 'C3', 'C4', 'C5')

Résumé

[label=●]Nombre total de combinaisons réalisables : 17 Nombre total de
combinaisons irréalisables : 14

Calcul du coût minimal pour chaque combinaison

Dans cette section, nous allons déterminer le coût minimal associé à chaque combinaison de clients. Nous supposons que chaque triporteur doit parcourir un certain nombre de clients dans un ordre déterminé afin de minimiser les coûts de livraison. Pour ce faire, nous utilisons une approche basée sur la minimisation des coûts de parcours, tout en respectant la capacité des triporteurs.

Formule du coût minimal

Soit $C_{i,j}$ le coût du trajet entre le client i et le client j . Le coût total pour une combinaison de clients donnée est calculé comme la somme des coûts de tous les trajets entre les clients dans la combinaison, ainsi que les trajets de la boulangerie vers le premier client et du dernier client vers la boulangerie. La formule générale du coût minimal est donc :

$$\text{Coût total}(S) = C_{B,i_1} + \sum_{k=1}^{n-1} C_{i_k,i_{k+1}} + C_{i_n,B}$$

où : - $S = \{i_1, i_2, \dots, i_n\}$ est l'ensemble des clients de la combinaison, - C_{B,i_1} est le coût du trajet entre la boulangerie B et le premier client i_1 , - $C_{i_k,i_{k+1}}$ est le coût du trajet entre les clients successifs, - $C_{i_n,B}$ est le coût du trajet du dernier client i_n vers la boulangerie.

Conditions de réalisabilité

Nous considérons qu'une combinaison est réalisable si le nombre de clients dans la combinaison ne dépasse pas la capacité maximale du triporteur. Si le nombre de clients dans une combinaison dépasse cette capacité, alors la combinaison est dite *irréalisable*.

Calcul du coût minimal

Nous avons calculé le coût minimal pour chaque combinaison réalisable en utilisant la formule ci-dessus. Voici les résultats obtenus pour le coût minimal de chaque combinaison réalisable :

Combinaison	Coût minimal
('C1',)	24
('C2',)	30
('C3',)	40
('C4',)	20
('C5',)	36
('C1', 'C2')	37
('C1', 'C3')	47
('C1', 'C4')	27
('C1', 'C5')	38
('C2', 'C3')	43
('C2', 'C4')	32
('C2', 'C5')	45
('C3', 'C4')	42
('C3', 'C5')	53
('C4', 'C5')	34
('C1', 'C2', 'C3')	50
('C1', 'C2', 'C4')	39
('C1', 'C2', 'C5')	47
('C1', 'C3', 'C4')	49
('C1', 'C3', 'C5')	55
('C1', 'C4', 'C5')	36
('C2', 'C3', 'C4')	45
('C2', 'C3', 'C5')	56
('C2', 'C4', 'C5')	43
('C3', 'C4', 'C5')	51
('C1', 'C2', 'C3', 'C4')	52
('C1', 'C2', 'C3', 'C5')	58
('C1', 'C2', 'C4', 'C5')	48
('C1', 'C3', 'C4', 'C5')	57
('C2', 'C3', 'C4', 'C5')	54
('C1', 'C2', 'C3', 'C4', 'C5')	60

Table 1: Coût minimal pour chaque combinaison de clients

Résumé des résultats

Le coût minimal pour chaque combinaison de clients est calculé en fonction des distances entre chaque paire de clients et des trajets nécessaires pour livrer tous les clients dans l'ordre optimal. Certaines combinaisons génèrent des coûts plus élevés que d'autres en raison des trajets plus longs ou de la complexité des tournées.

L'optimisation des tournées devrait permettre de réduire les coûts totaux en choisissant les combinaisons avec les coûts les plus bas. Cependant, certaines combinaisons sont irréalisables en raison des limites de capacité des triporteurs.

```
1 // Declarations
2 setof(int) Clients = {1, 2, 3, 4, 5}; // Clients
3 setof(int) positions = {0, 1, 2, 3, 4, 5}; // positions
4 setof(int) Vehicles = {1, 2, 3}; // Vehicules
5 float Capacity[Vehicles] = [250, 300, 200]; // Capacites des
   vehicules
6 float Demand[Clients] = [100, 150, 200, 100, 50]; //
   Demandes des clients
7 float Cost[0..5][0..5] = [
8   [0, 12, 15, 20, 10, 18], // Matrice des couts
9   [12, 0, 10, 15, 5, 8],
10  [15, 10, 0, 8, 7, 12],
11  [20, 15, 8, 0, 12, 15],
12  [10, 5, 7, 12, 0, 6],
13  [18, 8, 12, 15, 6, 0]
14 ];
15
16 // Variables de decision
17 dvar boolean x[0..5][0..5][Vehicles]; // 1 si le vehicule
   parcourt (i, j)
18 dvar float+ u[0..5][Vehicles]; // Charge transportee
   apres le client i
19
20 // Fonction objectif
21 minimize sum(k in Vehicles, i in 0..5, j in 0..5 : i != j)
   Cost[i][j] * x[i][j][k];
22
23 // Contraintes
24 subject to {
25   // Chaque client est visite une fois
26   forall(i in Clients)
27     sum(k in Vehicles, j in 0..5 : j != i) x[i][j][k] == 1;
28
29   // Flux entrant et sortant equilibre
30   forall(k in Vehicles, i in 0..5)
31     sum(j in 0..5 : j != i) x[i][j][k] == sum(j in 0..5 : j
   != i) x[j][i][k];
32
```

```

33 // Capacite respect e
34 forall(k in Vehicles, i in 0..5, j in 1..5 : i != j)
35     u[i][k] + Demand[j] <= u[j][k] + Capacity[k] * (1 - x[i
        ][j][k]);
36
37 // Limites sur u
38 forall(k in Vehicles, i in 1..5)
39     Demand[i] <= u[i][k] <= Capacity[k];
40 }
41
42 // Resolution
43 execute {
44     writeln("Cout total : ", cplex.getObjValue());
45     for (var k in Vehicles)
46         for (var i in positions)
47             for(var j in positions)
48                 if ( i != j)
49                     if (x[i][j][k] > 0.5) writeln("
                        Vehicule ", k, ": ", i, " -> ", j
                        );
50 }

```

0.2 Exercice 4:

question 1:

Dans le contexte de l'exercice 4, qui s'intéresse à la **vitesse de convergence des solutions et à l'impact des plans coupants (de type coupe de Chvátal-Gomory ou similaires), une réponse détaillée à la question 1.

Résolution sans ajouter de coupes

1. Principe : Désactiver les coupes dans le solveur Par défaut, les solveurs comme CPLEX utilisent un processus d'optimisation combinant la méthode de branchement et bornes (Branch-and-Bound) et des coupes automatiques pour réduire l'espace de recherche.

Pour répondre à la question 1, désactiver les coupes dans le solveur afin de résoudre le problème uniquement via le Branch-and-Bound. Cela permet de comparer la solution obtenue avec et sans coupes.

2. Désactivation des coupes dans CPLEX

Méthode 1 : Utiliser un fichier .opt pour configurer les paramètres Dans CPLEX, vous pouvez contrôler l'utilisation des coupes via un fichier .opt. Pour désactiver les coupes, ajoutez ces paramètres :

text CPLEX Cuts: Cliques -1 Covers -1 Disjunctive -1 FlowCovers -1 Flow-Paths -1 GUBCovers -1 ImpliedBound -1 Knapsack -1 LiftAndProject -1 MIR -1 ZeroHalf -1

Explication : - La valeur -1 désactive complètement une famille de coupes.
- Les familles listées incluent des coupes classiques comme les coupes de clique,

les coupes de couverture (Covers), et les coupes MIR.

img/M9_1.jpeg

Résultats attendus

1. Solution fractionnaire Le solveur déterminera une solution fractionnaire optimale à la relaxation linéaire initiale, avant de poursuivre avec le Branch-and-Bound pour rechercher une solution entière.

2. Solution optimale entière La solution optimale sera obtenue, mais avec un temps de convergence plus élevé et un plus grand nombre de nœuds explorés dans l'arbre de recherche (car les coupes ne sont pas utilisées pour réduire l'espace de recherche).

3. Indicateurs à rapporter Pour comparer les performances, vous devez noter : - La solution optimale fractionnaire obtenue après la relaxation linéaire. - La solution optimale entière obtenue après le Branch-and-Bound. - Le temps de résolution total (CPU Time). - Le nombre de nœuds explorés dans l'arbre de recherche.

Sortie attendue :

Solution fractionnaire (relaxation) : 123.45 Solution entière optimale : 130.00
Temps de résolution (s) : 25.4 Nombre de nœuds de branchement : 457

img/M9_2.jpeg

img/M9_3.jpeg

img/M9_4.jpeg

question 2 :

img/M9_5.jpeg

img/M9_6.jpeg

img/M9_7.jpeg

img/M9_8.jpeg

Bien que nous ayons utilisé les coupes de Gomory pour améliorer la convergence, le temps d'exécution a augmenté. Cela est dû au coût supplémentaire lié à la génération et au traitement des coupes, qui peut parfois surpasser les gains obtenus en réduisant l'espace de recherche. Cela montre que l'impact des coupes dépend fortement de la structure du problème et de sa taille.